

Assignment No. 2

Course Title: Parallel and Distributed Computing Lab (CS-426L)

JAVA OOP, THREADS, FORK/JOIN AND CACHE LOCALITY

Assignment Number 2

SUBMITTED BY:

Name: M. Dawood Nazir

Student ID: 247202

Semester: 5th

Section: BS-CYS-F-24-B

SUBMITTED TO:

Instructor Name: Mr. Qaiser Ali

Date of Submission: 25/10/2026

AIR UNIVERSITY, KHARIAN CAMPUS

DEPARTMENT OF COMPUTER SCIENCE

GitHub Repository: https://github.com/dawoodnazir3009/PDC-Lab-Assignment-2-Fall-2026

Question 1: Trading Engine Event Dispatch (Inheritance and Polymorphism)

Source Code: TradeEvent.java

```
package Q1;

public class TradeEvent {

    protected String SenderID;
    protected long timestamp;
    protected double price;



    TradeEvent(String SenderID, long timestamp, double price)
    {
        this.SenderID = SenderID;
        this.timestamp = timestamp;
        this.price = price;
    }

    public void dispatch()
    {
        System.out.println("Generic Trade Event");
        System.out.println("Sender ID is : " + SenderID);
        System.out.println("Timestamp is : " + timestamp);
        System.out.println("Base Price is : " + price);

    }

}
```

Source Code: FIXOrderEvent.java

```
package Q1;

public class FIXOrderEvent extends TradeEvent
{

    private static final double TRANSACTION_FEE_RATE = 0.0015; // 0.15%

        private String clOrdId;

        public FIXOrderEvent(String SenderID, long timestamp, double basePrice, String clOrdId) {
            super(SenderID, timestamp, basePrice);
            this.clOrdId = clOrdId;
        }

        @Override
        public void dispatch()
        {
            double fee = price * TRANSACTION_FEE_RATE;
            double totalcost = price + fee;
            System.out.println("[FIX ORDER] Executing stock purchase request");
            System.out.println("\tSender        : " + SenderID);
```

```
        System.out.println("\tClOrdID       : " + clOrdId);
        System.out.println("\tTimestamp     : " + timestamp);
        System.out.println("\tBase Price    : $" + String.format("%.2f", price));
        System.out.println("\tFee (0.15%)   : $" + String.format("%.4f", fee));
        System.out.println("\tTotal Cost    : $" + String.format("%.4f", totalcost));
        System.out.println("\tStatus        : ORDER EXECUTED successfully");

    }

}
```

Source Code: ReutersQuoteEvent.java

```
package Q1;

public class ReutersQuoteEvent extends TradeEvent {
   private double bidprice;
   private double askprice;

   public ReutersQuoteEvent(String SenderID, long timestamp, double price, double bidprice, double askprice)
   {
      super(SenderID, timestamp, price);
      this.bidprice = bidprice;
      this.askprice = askprice;
   }


   @Override
   public void dispatch()
   {
      double s = askprice - bidprice;
      double midprice = (askprice + bidprice) / 2.0;
      double spreadPCT = (s / midprice) * 100.0; //percentage

      System.out.println("[REUTERS QUOTE] Real-time bid-ask analytics");
      System.out.println("\tSender        : " + SenderID);
      System.out.println("\tTimestamp     : " + timestamp);
      System.out.println("\tBase Price    : $" + String.format("%.2f", price));
      System.out.println("\tBid / Ask     : $" + String.format("%.2f", bidprice) + " / $" + String.format("%.2f",
askprice));
      System.out.println("\tSpread        : $" + String.format("%.4f", s));
      System.out.println("\tMid Price     : $" + String.format("%.4f", midprice));
      System.out.println("\tSpread (%)    : " + String.format("%.4f", spreadPCT) + "% ");



   }

}
```

Source Code: TradingEngineApp.java

```
package Q1;

public class TradingEngineApp
{
```

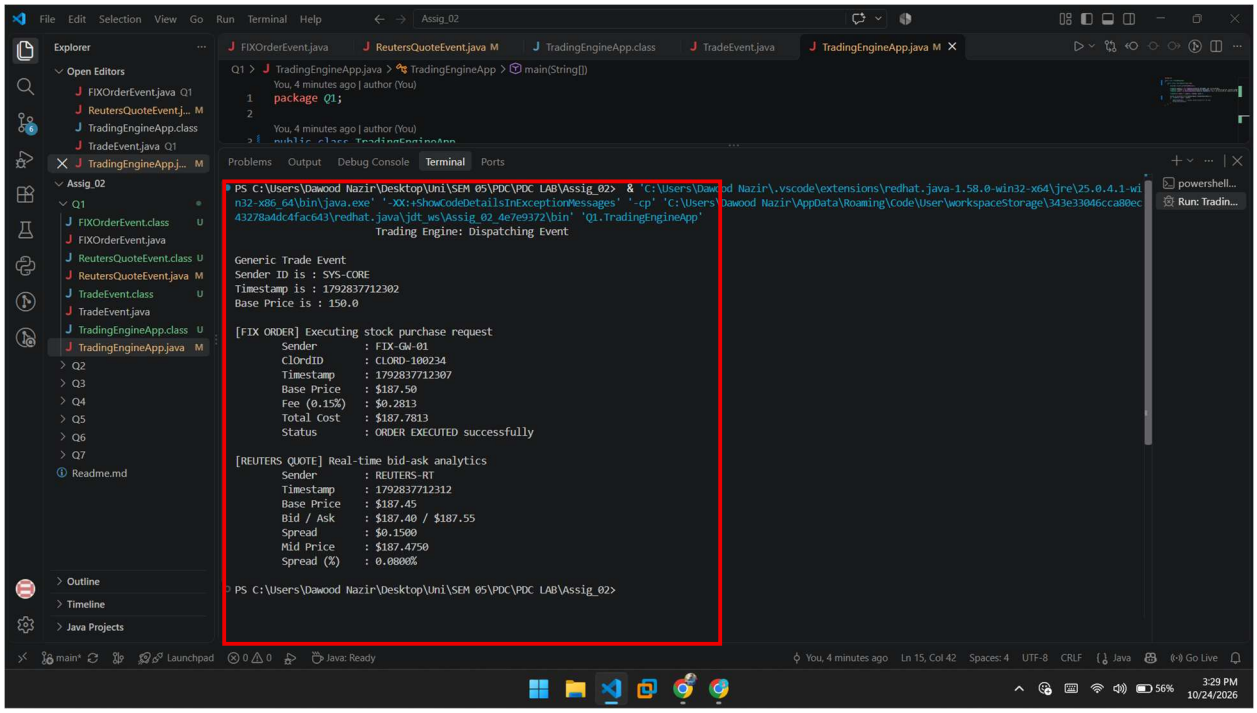
```
    public static void main(String[] args)
    {
        long now = System.currentTimeMillis();

        TradeEvent generic = new TradeEvent("SYS-CORE", now, 150.00);
        TradeEvent fixOrder = new FIXOrderEvent("FIX-GW-01", now + 5, 187.50, "CLORD-100234");
        TradeEvent quote = new ReutersQuoteEvent("REUTERS-RT", now + 10, 187.45, 187.40, 187.55);

        TradeEvent[] events = { generic, fixOrder, quote };

        System.out.println("\t\t\tTrading Engine: Dispatching Event\n");
        for (TradeEvent event : events)
        {
            event.dispatch();   // dynamic method dispatch at run time
            System.out.println();
        }
    }
}
```

Execution Screenshot



Approach and Explanation

To solve this task I modelled a trading event hierarchy with four classes, each placed in its own .java file inside package Q1. TradeEvent.java is the superclass; it declares the protected fields SenderID, timestamp and price so that subclasses can access them directly, a parameterized constructor that assigns all three fields with the this keyword, and a baseline dispatch() method that prints the generic event details. FIXOrderEvent.java extends TradeEvent, adds a private clOrdId variable and a private static final constant TRANSACTION_FEE_RATE = 0.0015 (the

0.15% exchange fee), calls super(SenderID, timestamp, basePrice) as the first statement of its constructor to reuse the parent initialization, and overrides dispatch() with the @Override annotation to compute fee = price * TRANSACTION_FEE_RATE and totalcost = price + fee before printing the formatted stock purchase report with String.format. ReutersQuoteEvent.java also extends TradeEvent, adds the private bidprice and askprice variables, and overrides dispatch() to compute the spread (askprice - bidprice), the mid price ((askprice + bidprice) / 2.0) and the spread percentage ((spread / midprice) * 100). In TradingEngineApp.java the main() method reads System.currentTimeMillis() once, creates a plain TradeEvent, a FIXOrderEvent (timestamp now + 5, base price 187.50, order ID CLORD-100234) and a ReutersQuoteEvent (timestamp now + 10, price 187.45, bid 187.40, ask 187.55), stores all three objects in one TradeEvent[] array and calls event.dispatch() on every element inside an enhanced for loop. Because Java selects the overridden method at run time from the actual object type and not from the reference type (dynamic method dispatch), the same call executed a different version of dispatch() for each element, which is runtime polymorphism and message passing through a superclass reference. After compiling from the repository root with javac Q1/*.java and running java Q1.TradingEngineApp, the terminal printed the generic event first, then the FIX order with a fee of $0.2813 and a total cost of $187.7813 (187.50 x 0.0015 = 0.28125), and finally the Reuters quote with a spread of $0.1500, a mid price of $187.4750 and a spread of 0.0800%; these values matched my manual calculations and confirmed that method overriding, the super constructor call and polymorphic dispatch all worked as intended.

Question 2: Packet Symmetry Auditor (Array-Based Stack and Queue)

Source Code: TransactionStack.java

```
package Q2;

public class TransactionStack
{
   private final char[] data;
   private int top;           // index of the top element; -1 when empty
   private final int capacity;


   public TransactionStack(int capacity)
   {

      this.capacity = capacity;
      this.data = new char[capacity];
      this.top = -1; //-1 as stack is currently empty
   }


   public boolean isEmpty()
   {
      if (top == -1)
         return true; //true as stack is empty
      else
         return false; // as stack is not empty
   }

   public boolean isFull()
   {

      if (top == (capacity -1))
         return true; //true as stack is full
      else
         return false; // as stack contains space
   }


   public boolean push(char c)
   {
      if (isFull())
      {
         System.out.println("STACK OVERFLOW! Cannot push \'" + c + "\'");
         return false;
      }
      //adding to stack
      top++;
      data[top] = c;
      return true; //successfully added
   }


   public char pop()
```

```
    {
        if (isEmpty())
        {
            System.out.println("!STACK UNDERFLOW! Nothing to pop");
            return '\0'; //line end caharcer
        }
        return data[top--];
    }


    public char peek()
    {
        if (isEmpty())
        {
            System.out.println("STACK EMPTY! Nothing to peek");
            return '\0';
        }
        return data[top];
    }
}
```

Source Code: Requestqueue.java

```
package Q2;

public class Requestqueue
{
    private final char[] data;
    private int front;   // index of the next element to dequeue
    private int rear;    // index where the next element will be enqueued
    private int size;    // current number of elements
    private final int capacity;

    public Requestqueue(int capacity)
    {
        this.capacity = capacity;
        this.data = new char[capacity];
        this.front = 0;
        this.rear = 0;
        this.size = 0;
    }

    public boolean isEmpty()
    {
        if (size == 0)
            return true;
        else
            return false;

    }

    public boolean isFull()
    {
        return size == capacity;
    }
```

```
    // Adds characyer "c" at the rear
    public boolean enqueue(char c)
    {
        if (isFull())
        {
            System.out.println("QUEUE OVERFLOW! Cannot enqueue \'" + c + "\'");
            return false;
        }
        data[rear] = c;
        rear = (rear + 1) % capacity;
        size++;
        return true;
    }

    // Removes and returns the front element dequeue
    public char dequeue()
    {
        if (isEmpty())
        {
            System.out.println("QUEUE UNDERFLOW! Nothing to dequeue");
            return '\0';
        }
        char c = data[front];
        front = (front + 1) % capacity;
        size--;
        return c;
    }




}
```

Source Code: PacketSymmetryAuditor.java

```
package Q2;
import  java.util.Scanner;

public class PacketSymmetryAuditor
{

    public PacketSymmetryAuditor()
    {



    }
/** Keeps only letters/digits and lower-cases them (ignores separators/case). */
    private static String normalize(String raw)
    {
        StringBuilder sb = new StringBuilder();
        for (int i = 0; i < raw.length(); i++)
        {
            char c = raw.charAt(i);
            if (Character.isLetterOrDigit(c))
```

```
            {
                sb.append(Character.toLowerCase(c));
            }
        }
        return sb.toString();
    }

    private static void audit(String rawPayload)
    {
        String payload = normalize(rawPayload);
        int n = payload.length();

        System.out.println("-----------------------------------------------");
        System.out.println("[AUDIT] Raw payload       : \"" + rawPayload + "\"");
        System.out.println("[AUDIT] Normalized payload: \"" + payload + "\"");

        if (n == 0)
        {
            System.out.println("[RESULT] Empty payload - nothing to verify.");
            return;
        }

        TransactionStack stack = new TransactionStack(n);
        Requestqueue queue = new Requestqueue(n);

        // Ingest: push into the stack while simultaneously enqueuing.
        for (int i = 0; i < n; i++)
        {
            char c = payload.charAt(i);
            stack.push(c);
            queue.enqueue(c);
        }

        // Verify: pop and dequeue together, comparing each pair.
        boolean symmetric = true;
        int step = 1;
        while (!stack.isEmpty() && !queue.isEmpty()) //iterate until both contains elements
        {
            char fromStack = stack.pop();       // last-in  (reverse stream)
            char fromQueue = queue.dequeue();   // first-in (forward stream)
            boolean match = (fromStack == fromQueue);
            System.out.println("  Step " + step++ + ": POP='" + fromStack
                    + "'  DEQUEUE='" + fromQueue + "'  -> " + (match ? "MATCH" : "MISMATCH"));
            if (!match) {
                symmetric = false;
                break;
            }
        }

        System.out.println(symmetric ? "[RESULT] VERIFIED: packet exhibits bi-directional symmetry
(palindrome signature)." : "[RESULT] REJECTED: packet is NOT symmetric.");
    }

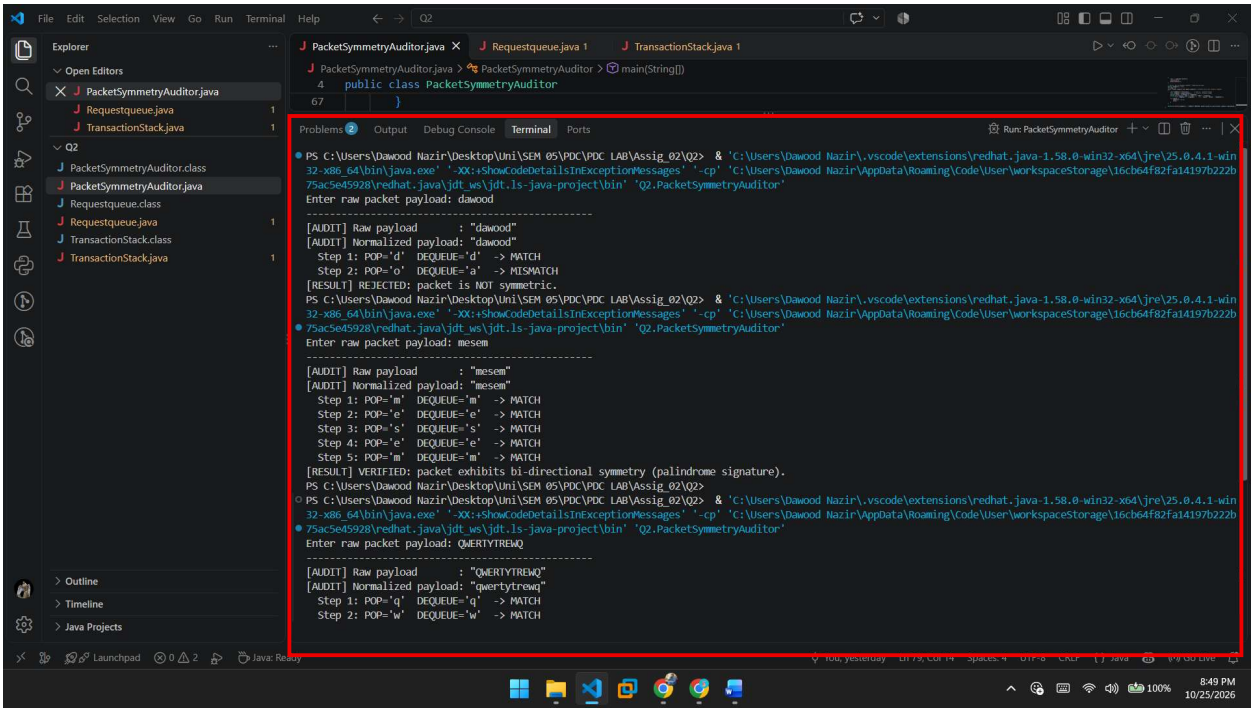
    public static void main(String[] args)
    {
        if (args.length > 0)
```

```
        {
            for (String payload : args)
            {
                audit(payload);
            }
            return;
        }

        Scanner in = new Scanner(System.in);
        System.out.print("Enter raw packet payload: ");
        String payload = in.hasNextLine() ? in.nextLine() : "";
        audit(payload);
        in.close();
    }

}
```

Execution Screenshot



Approach and Explanation

I solved this task by first building two custom array-based data structures without using any java.util collection and then combining them inside an auditor class, with every class in its own file inside package Q2. TransactionStack.java stores char values in a fixed-size char[] whose capacity is passed to the constructor and keeps an int variable top initialized to -1 to represent an empty stack; isEmpty() tests top == -1, isFull() tests top == capacity - 1, push(char c) prints STACK OVERFLOW when full and otherwise increments top before storing the character, pop() prints an underflow message when empty and otherwise returns data[top--], and peek() returns data[top] without changing the pointer. Requestqueue.java implements a circular queue on a char[]

with the integer variables front, rear and size; enqueue() writes at data[rear] and advances rear = (rear + 1) % capacity, dequeue() reads data[front] and advances front in the same way, and isEmpty() and isFull() use size to report overflow and underflow. In PacketSymmetryAuditor.java the private static normalize() method loops over the raw payload with charAt(i), keeps only the characters accepted by Character.isLetterOrDigit() and converts them with Character.toLowerCase() so that spaces, punctuation and letter case are ignored, and the audit() method creates a TransactionStack and a Requestqueue of the normalized length n and uses one for loop to push and enqueue every character at the same time. A while loop then runs as long as both structures are non-empty, popping the last-in character from the stack (the reversed stream) and dequeuing the first-in character from the queue (the forward stream), printing each pair as MATCH or MISMATCH and breaking at the first mismatch, so the packet is verified only when every pair is equal. The main() method audits every command-line argument, or reads one payload with a Scanner when no argument is given. Running java Q2.PacketSymmetryAuditor with the payloads "A man, a plan, a canal: Panama" and "Hello" showed the first payload normalized to amanaplanacanalpanama and produced 21 consecutive MATCH steps followed by the VERIFIED message, while the second payload normalized to hello and was REJECTED at step 1 because POP returned 'o' and DEQUEUE returned 'h', which proved that the LIFO stack and the FIFO queue behave as exact opposites and that the symmetry check works.

Question 3: Asynchronous Web Engine Using Runnable and Thread

Source Code: LogProcessorTask.java

```
package Q3;


public class LogProcessorTask implements Runnable
{

   @Override
   public void run()
   {
      for (int i = 1; i <= 5; i++)
      {
         System.out.println("[" + Thread.currentThread().getName() + "] "
               + "Log analysis iteration " + i + "/5 - processed batch "
               + (i * 250) + " log entries");

      try {
            Thread.sleep(1000);
         }
         catch (InterruptedException e)
         {
            System.out.println("[" + Thread.currentThread().getName() + "] Thread was interrupted.");
            Thread.currentThread().interrupt();
            break;
         }

      }
      System.out.println("[" + Thread.currentThread().getName() + "] Log processing complete.\n");
   }
}
```

Source Code: TelemetryThread.java

```
package Q3;

public class TelemetryThread extends Thread
{

   @Override
   public void run()
   {
      Runtime rt = Runtime.getRuntime();

      for (int i = 1; i <= 5; i++)
      {
         long usedMb = (rt.totalMemory() - rt.freeMemory()) / (1024 * 1024);
         long maxMb = rt.maxMemory() / (1024 * 1024);
         double load =
java.lang.management.ManagementFactory.getOperatingSystemMXBean().getSystemLoadAverage();
         String cpu;
         if  (load < 0)
            cpu = "n/a";
```

```
            else
                cpu = String.format("%.2f", load);
            int cores = rt.availableProcessors();
            System.out.print("[" + getName() + "] Health update " + i + "/5");
            System.out.print(" | CPU cores: " + cores + " | CPU load avg: " + cpu);
            System.out.println(" | Memory: " + usedMb + "MB / " + maxMb + "MB");

            try
            {
                Thread.sleep(800);
            }
            catch (InterruptedException e)
            {
                System.out.println("[" + getName() + "] Interrupted.");
                Thread.currentThread().interrupt();
                return;
            }
        }
        System.out.println("[" + getName() + "] Telemetry reporting complete.");
    }
}
```

Source Code: AsyncWebEngineApp.java

```
package Q3;

public class AsyncWebEngineApp
{

    public static void main(String[] args)
    {
        // Default JVM thread name
        System.out.println("[main] Default JVM thread name: " + Thread.currentThread().getName());

        // Thread created by extending Thread
        TelemetryThread telemetry = new TelemetryThread();
        telemetry.setName("Telemetry-Worker");

        // Thread created by passing a Runnable
        Thread logWorker = new Thread(new LogProcessorTask());
        logWorker.setName("Log-Worker");

        System.out.print("[main] Before start -> " + telemetry.getName() + ": ");
        System.out.print( telemetry.getState() + " | " + logWorker.getName() + ": ");
        System.out.println(logWorker.getState() );

        // Start both threads back to back so they run concurrently
        telemetry.start();
        logWorker.start();

        System.out.print("[main] After start  -> " + telemetry.getName() + ": " + telemetry.getState());
        System.out.print(" (alive=" + telemetry.isAlive() + ") | " + logWorker.getName() + ": ");
        System.out.println(logWorker.getState() + " (alive=" + logWorker.isAlive() + ")");

        // main() waits for both workers to exit cleanly
```

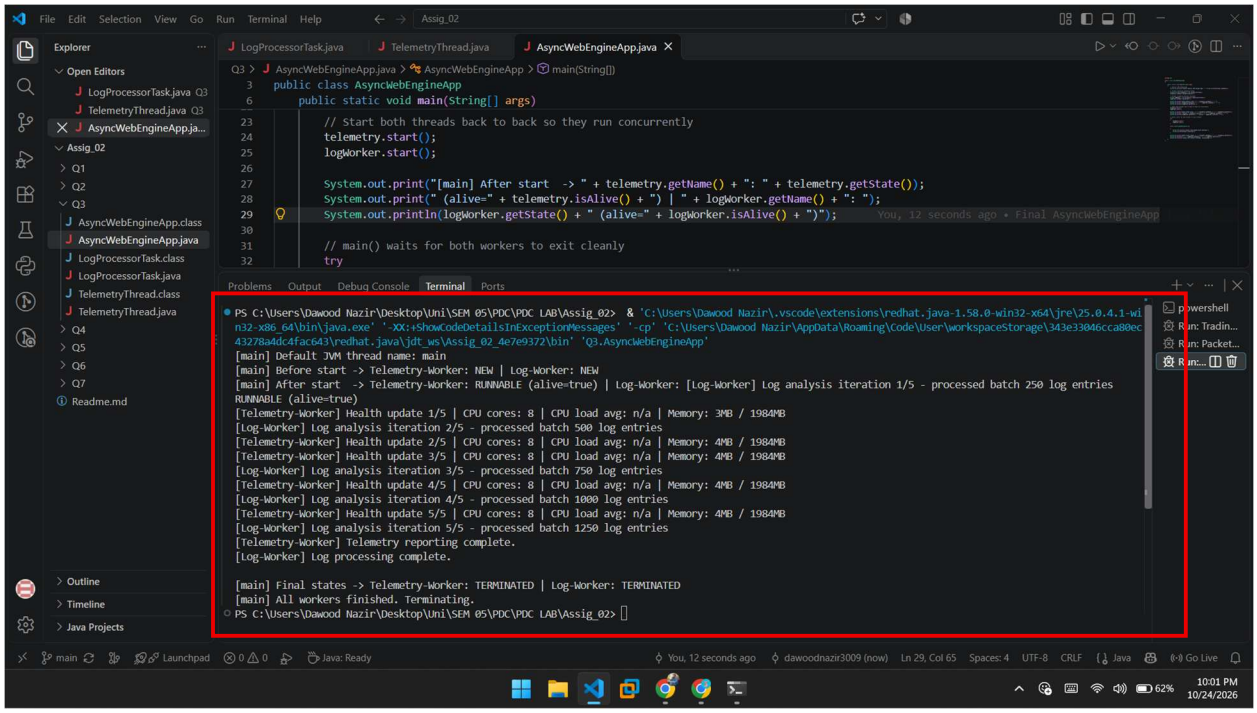
```
        try
        {
            telemetry.join();
            logWorker.join();
        }

        catch (InterruptedException e)

        {
            System.out.println("[main] Interrupted while waiting.");
            Thread.currentThread().interrupt();
        }

        System.out.print("[main] Final states -> " + telemetry.getName() + ": " + telemetry.getState());
        System.out.println(" | " + logWorker.getName() + ": " + logWorker.getState());
        System.out.println("[main] All workers finished. Terminating.");
    }
}
```

Execution Screenshot



Approach and Explanation

I demonstrated the two standard ways of creating threads in Java by using three separate files inside package Q3. LogProcessorTask.java implements the Runnable interface and overrides run() with a for loop of five iterations; each iteration prints Thread.currentThread().getName(), the iteration number and a processed batch size of i * 250 log entries, and then calls Thread.sleep(1000) inside a try-catch block that handles InterruptedException by restoring the interrupt flag and leaving the loop. TelemetryThread.java extends the Thread class directly and

overrides run() with another five-iteration loop; in every iteration it reads Runtime.getRuntime() to print the number of available CPU cores, the system load average (shown as n/a when the operating system does not report one) and the used and maximum heap memory in MB, and then pauses with Thread.sleep(800) so that it runs faster than the log task. In AsyncWebEngineApp.java the main() method first prints Thread.currentThread().getName(), which shows the default JVM name main, then creates a TelemetryThread object and a new Thread that receives a LogProcessorTask instance in its constructor, assigns the custom names Telemetry-Worker and Log-Worker with setName(), and prints their states with getState() before starting them. It then calls start() on both threads back to back so that they run concurrently, prints the states again together with isAlive(), and finally calls join() on both threads inside a try-catch block so that main waits until both workers finish before printing the final states. When I compiled with javac Q3/*.java and ran java Q3.AsyncWebEngineApp, the terminal showed the main thread name, the state NEW for both threads before start and RUNNABLE with alive=true after start, and then the Telemetry-Worker and Log-Worker messages interleaved on the console because their sleep intervals of 800 milliseconds and 1000 milliseconds differ; the Telemetry-Worker finished its five updates (about 4 seconds) before the Log-Worker finished its five iterations (about 5 seconds), and the TERMINATED states printed after join() confirmed that both threads completed all of their work.

Question 4: Fork/Join Portfolio Assessment Engine

Source Code: ParallelRiskCalculator.java

```
package Q4;
import java.util.concurrent.RecursiveTask;

public class ParallelRiskCalculator extends RecursiveTask<Double>
{
    private static final int THRESHOLD = 1000;

    private final double[] assets;
    private final int start;   // inclusive
    private final int end;     // exclusive

    public ParallelRiskCalculator(double[] assets, int start, int end)
    {
        this.assets = assets;
        this.start = start;
        this.end = end;
    }

    @Override
    protected Double compute()
    {
        int length = end - start;


        if (length <= THRESHOLD)
        {
            double sum = 0.0;

            int i = start;
            for (; i < end; i++)
            {
                sum += assets[i];
            }
            return sum;
        }


        int mid = start + length / 2;
        ParallelRiskCalculator left = new ParallelRiskCalculator(assets, start, mid);
        ParallelRiskCalculator right = new ParallelRiskCalculator(assets, mid, end);

        left.fork();
        right.fork();

        double leftResult = left.join();
        double rightResult = right.join();

        return leftResult + rightResult;
    }
}
```

Source Code: PortfolioThresholdTask.java

```
package Q4;

import java.util.concurrent.RecursiveAction;

public class PortfolioThresholdTask extends RecursiveAction
{
   private static final int THRESHOLD = 1000;
   private static final double PENALTY = 0.05; //5 percent

   private final double[] assets;
   private final int start;
   private final int end;
   private final double riskCap;


    public PortfolioThresholdTask (double[] assets, int start, int end, double riskCap)
   {
      this.assets = assets;
      this.start = start;
      this.end = end;
      this.riskCap = riskCap;
   }


   @Override
   protected void compute()
   {
      int length = end - start;

      if (length <= THRESHOLD)
      {
         for (int i = start; i < end; i++)
         {
            if (assets[i] > riskCap)
            {
               assets[i] -= assets[i] * PENALTY;   // apply 5% penalty
            }
         }
         return;
      }

      int mid = start + length / 2;
      PortfolioThresholdTask left = new PortfolioThresholdTask(assets, start, mid, riskCap);
      PortfolioThresholdTask right = new PortfolioThresholdTask(assets, mid, end, riskCap);

      // Fork both halves and wait for them to finish
      invokeAll(left, right);
   }
}
```

Source Code: ForkJoinPortfolioApp.java

```
package Q4;
```

```
import java.util.Random;
import java.util.concurrent.ForkJoinPool;


public class ForkJoinPortfolioApp
{
   private static final double RISK_CAP = 800.0;

   private static double serialSum(double[] a)
   {
      double sum = 0.0;
      for (double v : a)
      {
         sum += v;
      }
      return sum;
   }

   private static void serialStress(double[] a)
   {
      for (int i = 0; i < a.length; i++)
      {
         if (a[i] > RISK_CAP)
         {
            a[i] -= a[i] * 0.05;
         }
      }
   }

   public static void main(String[] args)
   {
      int size;
      if (args.length > 0)
         size = Integer.parseInt(args[0]);
      else
         size = 10_000_000;

      // Generate synthetic asset values in [10, 1000)
      Random rnd = new Random(42);
      double[] original = new double[size];
      for (int i = 0; i < size; i++)
      {
         original[i] = 10.0 + rnd.nextDouble() * 990.0;
      }

      ForkJoinPool pool = new ForkJoinPool();   // parallelism = available cores
      System.out.println("--- Parallel Portfolio Assessment Engine ---");
      System.out.print("Assets: " + size + " | Risk cap: " + RISK_CAP);
      System.out.println(" | Pool parallelism: " + pool.getParallelism());

      // ---- Single-run results (correctness) ----
      double serialRisk = serialSum(original);
      double parallelRisk = pool.invoke(new ParallelRiskCalculator(original, 0, size));

      double[] serialStressed = original.clone();
```

```
        serialStress(serialStressed);

        double[] parallelStressed = original.clone();
        pool.invoke(new PortfolioThresholdTask(parallelStressed, 0, size, RISK_CAP));

        int penalized = 0;
        for (double v : original)
        {
            if (v > RISK_CAP) penalized++;
        }
        double serialStressedRisk = serialSum(serialStressed);
        double parallelStressedRisk = pool.invoke(new ParallelRiskCalculator(parallelStressed, 0, size));

        System.out.printf("%nCumulative risk (serial)   : %.4f%n", serialRisk);
        System.out.printf("Cumulative risk (parallel) : %.4f%n", parallelRisk);
        System.out.println("Assets above cap (penalized): " + penalized);
        System.out.printf("Post-stress risk (serial)  : %.4f%n", serialStressedRisk);
        System.out.printf("Post-stress risk (parallel): %.4f%n", parallelStressedRisk);
        System.out.println("Results match: "
                + (Math.abs(serialRisk - parallelRisk) < 1e-6 * serialRisk
                && Math.abs(serialStressedRisk - parallelStressedRisk) < 1e-6 * serialStressedRisk));


        int reps = Math.max(5, 2_000_000 / size);
        for (int w = 0; w < 50; w++)
        {     // warm-up
            serialSum(original);
            pool.invoke(new ParallelRiskCalculator(original, 0, size));
            double[] t1 = original.clone();
            serialStress(t1);
            double[] t2 = original.clone();
            pool.invoke(new PortfolioThresholdTask(t2, 0, size, RISK_CAP));
        }

        long serialNs = 0, parallelNs = 0;
        for (int r = 0; r < reps; r++)
        {
            double[] s = original.clone();
            double[] p = original.clone();

            long t = System.nanoTime();
            serialSum(s);
            serialStress(s);
            serialNs += System.nanoTime() - t;

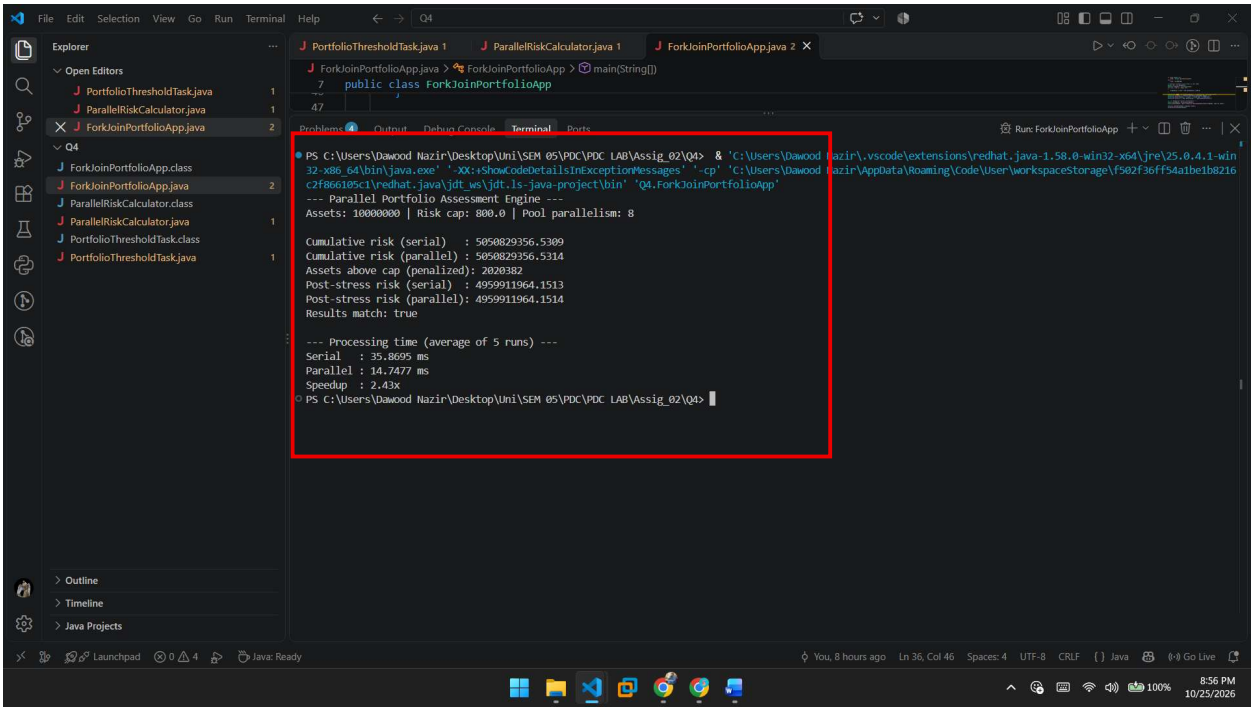
            t = System.nanoTime();
            pool.invoke(new ParallelRiskCalculator(p, 0, size));
            pool.invoke(new PortfolioThresholdTask(p, 0, size, RISK_CAP));
            parallelNs += System.nanoTime() - t;
        }

        double serialMs = serialNs / 1e6 / reps;
        double parallelMs = parallelNs / 1e6 / reps;
        System.out.println("\n--- Processing time (average of " + reps + " runs) ---");
        System.out.printf("Serial   : %.4f ms%n", serialMs);
        System.out.printf("Parallel : %.4f ms%n", parallelMs);
```

```
        System.out.printf("Speedup  : %.2fx%n", serialMs / parallelMs);

        pool.shutdown();
    }
}
```

Execution Screenshot



Approach and Explanation

To build the parallel portfolio engine I used the Fork/Join framework with three classes inside package Q4. ParallelRiskCalculator.java extends RecursiveTask<Double> and stores the asset array together with the start (inclusive) and end (exclusive) indexes of its segment; its compute() method returns a direct loop sum when the segment length is at most the THRESHOLD constant of 1000, and otherwise finds mid = start + length / 2, creates a left and a right ParallelRiskCalculator, calls fork() on both and then join() on both to add the two partial sums, which is a divide-and-conquer reduction. PortfolioThresholdTask.java extends RecursiveAction, uses the same threshold of 1000 and a PENALTY constant of 0.05, and its compute() method either loops over a small segment and applies assets[i] -= assets[i] * PENALTY to every value above the riskCap, or splits the segment in half and runs both halves with invokeAll(left, right); because it modifies the array in place it returns no value. In ForkJoinPortfolioApp.java the main() method sets RISK_CAP to 800.0, uses 10,000,000 assets by default (or the size given as the first command-line argument), and fills the array from new Random(42) with values of 10.0 + rnd.nextDouble() * 990.0 so that every run uses the same data. It creates a ForkJoinPool whose parallelism equals the number of available cores and prints it with getParallelism(), computes the cumulative risk serially with serialSum() and in parallel with pool.invoke(new

ParallelRiskCalculator(original, 0, size)), applies the stress test serially and in parallel on cloned arrays, counts the assets above the cap, and prints the post-stress risk for both versions. Because the parallel version adds the numbers in a different order, the correctness check compares the results with a relative tolerance of 1e-6 instead of exact equality. For the timing section I ran 50 warm-up iterations so the JIT compiler could optimize the code, then measured Math.max(5, 2,000,000 / size) repetitions with System.nanoTime() on fresh clones of the array and printed the average serial time, the average parallel time and the speedup (serial divided by parallel). After compiling with javac Q4/*.java and running java Q4.ForkJoinPortfolioApp, the serial and parallel cumulative risk and post-stress risk values agreed and the program printed Results match: true, which verified that the RecursiveTask and RecursiveAction implementations compute the same answers as the sequential loops, while the printed speedup shows the effect of the available cores and the task-splitting overhead on this machine.

Question 5: IoT Sensor Grid Monitor (Abstract Class and Parallel Search)

Source Code: SensorSearchEngine.java

```
package Q5;

public abstract class SensorSearchEngine
{
   protected double[] sensorData;
   protected int startIdx, endIdx, numThreads;
   protected double result;
   protected long elapseNanos;


   SensorSearchEngine(double[] sensorData, int startIdx, int endIdx, int numThreads)
   {
      if (startIdx>=endIdx)
         throw new IllegalArgumentException("Empty Sensor Segment");

      this.startIdx = startIdx;
      this.endIdx = endIdx;
      this.sensorData = sensorData;
      this.numThreads = numThreads;
   }

   public abstract void processGrid();

   // getters
   public double getResult()
   {
      return result;
   }

   public double getElapseMillis()
   {
      return elapseNanos / 1e6;
   }

   protected int[] partitionBounds()
   {

      int length = endIdx - startIdx;
      int chunk = length / numThreads;
      int[] bounds = new int[numThreads + 1];

      for(int i =0; i < numThreads; i++)
      {

         bounds[i] = startIdx + i * chunk;
      }

      bounds[numThreads] = endIdx;
      return bounds;

   }
```

```
}
```

Source Code: ParallelMinSearch.java

```
package Q5;

public class ParallelMinSearch extends SensorSearchEngine
{
   public ParallelMinSearch(double[] sensorData, int startIdx, int endIdx, int numThredas)
   {
      super(sensorData, startIdx, endIdx, numThredas);
   }

   @Override
   public void processGrid()
   {
      long t0 = System.nanoTime();
      int[] bounds = partitionBounds();
      double[] localMin = new double[numThreads];
      Thread[] workers = new Thread[numThreads];
      for (int i = 0; i < numThreads; i++)
      {
         final int id = i;
         final int from = bounds[i];
         final int to = bounds[i + 1];
         workers[i] = new Thread(() -> {double min = Double.MAX_VALUE;
            for (int j = from; j < to; j++)
            {
               if (sensorData[j] < min)
               {
                  min = sensorData[j];
               }
            }
            localMin[id] = min;
            System.out.printf("  [%s] segment [%d..%d) local min temperature = %.3f%n",
                  Thread.currentThread().getName(), from, to, min);
         }, "MinWorker-" + (i + 1));
         workers[i].start();
      }

      // Wait for all workers, then reduce the partial results
      double globalMin = Double.MAX_VALUE;
      for (int i = 0; i < numThreads; i++)
      {
         try
         {
            workers[i].join();
         }
         catch (InterruptedException e)
         {
            Thread.currentThread().interrupt();
```

```
            return;
        }
        globalMin = Math.min(globalMin, localMin[i]);
    }

    result = globalMin;
    elapseNanos = System.nanoTime() - t0;
  }
}
```

Source Code: ParallelMaxSearch.java

```
package Q5;

public class ParallelMaxSearch extends SensorSearchEngine
{
  public ParallelMaxSearch(double[] sensorData, int startIdx, int endIdx, int numThreads) {
    super(sensorData, startIdx, endIdx, numThreads);
  }

  @Override
  public void processGrid()
  {
    long t0 = System.nanoTime();
    int[] bounds = partitionBounds();
    double[] localMax = new double[numThreads];
    Thread[] workers = new Thread[numThreads];

    for (int i = 0; i < numThreads; i++)
    {
      final int id = i;
      final int from = bounds[i];
      final int to = bounds[i + 1];
      workers[i] = new Thread(() -> {double max = -Double.MAX_VALUE;
        for (int j = from; j < to; j++)
        {
          if (sensorData[j] > max)
          {
            max = sensorData[j];
          }
        }
        localMax[id] = max;
        System.out.printf("  [%s] segment [%d..%d) local peak vibration =
%.3f%n",Thread.currentThread().getName(), from, to, max);
      }, "MaxWorker-" + (i + 1));

      workers[i].start();
    }

    // Wait for all workers
    double globalMax = -Double.MAX_VALUE;

    for (int i = 0; i < numThreads; i++)
    {
      try
```

```
            {
                workers[i].join();
            }
            catch (InterruptedException e)
            {
                Thread.currentThread().interrupt();
                return;
            }
            globalMax = Math.max(globalMax, localMax[i]);
        }

        result = globalMax;
        elapseNanos = System.nanoTime() - t0;

    }

}
```

Source Code: SensorGridMonitorApp.java

```
package Q5;
import java.util.Random;

public class SensorGridMonitorApp
{
    private static final int SENSOR_COUNT = 50_000;
    private static final int WORKERS = 4;

    public static void main(String[] args)
    {


        Random rnd = new Random(7);
        double[] telemetry = new double[SENSOR_COUNT];

        for (int i =0; i < SENSOR_COUNT; i++)
        {
            telemetry[i] = 20.0 + rnd.nextDouble() * 60.0;
        }

        telemetry[12_345] = -12.5;
        telemetry[37_890] = 312.8;

        System.out.println("--- IoT Sensor Grid Monitor ---");
        System.out.println("Sensor nodes: " + SENSOR_COUNT + " | Worker threads per search: " + WORKERS);

        // Serial reference for verification and timing
        long s0 = System.nanoTime();
        double serialMin = Double.MAX_VALUE;
        double serialMax = -Double.MAX_VALUE;
        for (double v : telemetry)
        {
            if (v < serialMin) serialMin = v;
            if (v > serialMax) serialMax = v;
        }
```

```
        long serialNs = System.nanoTime() - s0;

        SensorSearchEngine minSearch = new ParallelMinSearch(telemetry, 0, SENSOR_COUNT, WORKERS);
        SensorSearchEngine maxSearch = new ParallelMaxSearch(telemetry, 0, SENSOR_COUNT, WORKERS);

        // Run both reductions simultaneously, each on its own coordinator thread
        Thread minRunner = new Thread(minSearch::processGrid, "MinSearch-Coordinator");
        Thread maxRunner = new Thread(maxSearch::processGrid, "MaxSearch-Coordinator");

        System.out.println("\n[main] Launching min and max reductions simultaneously...");
        long p0 = System.nanoTime();
        minRunner.start();
        maxRunner.start();
        try
        {
           minRunner.join();
           maxRunner.join();
        }

        catch (InterruptedException e)
        {
           Thread.currentThread().interrupt();
           return;
        }
        long parallelNs = System.nanoTime() - p0;

        System.out.println("\n--- Global Critical Thresholds ---");
        System.out.printf("Minimum temperature (parallel): %.3f%n", minSearch.getResult());
        System.out.printf("Peak vibration spike (parallel): %.3f%n", maxSearch.getResult());
        System.out.println("Matches serial reference: "
              + (minSearch.getResult() == serialMin && maxSearch.getResult() == serialMax));

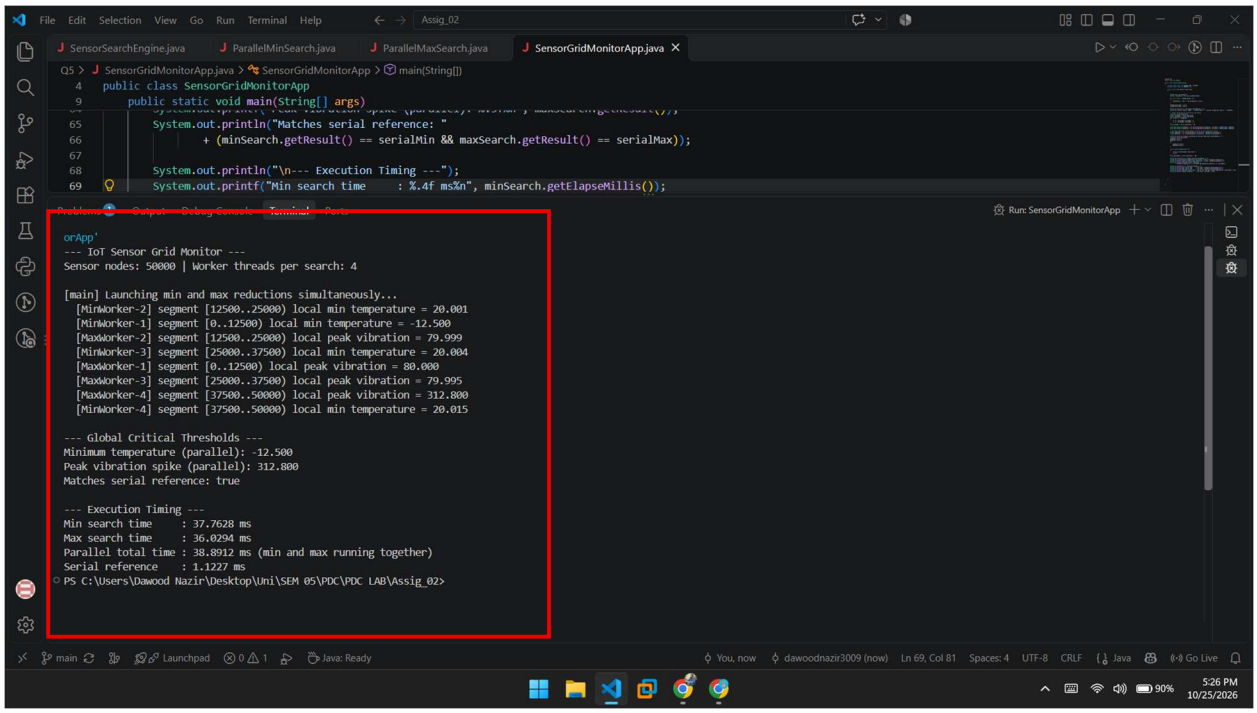
        System.out.println("\n--- Execution Timing ---");
        System.out.printf("Min search time     : %.4f ms%n", minSearch.getElapseMillis());
        System.out.printf("Max search time     : %.4f ms%n", maxSearch.getElapseMillis());
        System.out.printf("Parallel total time : %.4f ms (min and max running together)%n", parallelNs / 1e6);
        System.out.printf("Serial reference    : %.4f ms%n", serialNs / 1e6);




    }


}
```

Execution Screenshot



Approach and Explanation

I solved this task with an abstract base class and two concrete parallel search classes, each in its own file inside package Q5. SensorSearchEngine.java declares the protected variables sensorData, startIdx, endIdx, numThreads, result and elapseNanos, a constructor that throws an IllegalArgumentException when startIdx >= endIdx, the abstract method processGrid(), the getters getResult() and getElapseMillis(), and a protected partitionBounds() method that divides the range into numThreads equal chunks (chunk = length / numThreads) and lets the last segment end at endIdx so that no element is skipped. ParallelMinSearch.java and ParallelMaxSearch.java extend this class, call super(...) in their constructors and override processGrid() with @Override: each one records System.nanoTime(), gets the segment bounds, creates one Thread per segment with a lambda Runnable named MinWorker-i or MaxWorker-i that scans only its own range [from, to) and stores its local minimum or maximum in its own slot of a double[] array (so the workers never write to the same location and no synchronization is needed), starts all workers, then calls join() on each of them and reduces the partial results with Math.min or Math.max before saving the result and the elapsed time. In SensorGridMonitorApp.java the main() method fills 50,000 sensor readings with new Random(7) in the range 20 to 80, plants one anomaly of -12.5 at index 12,345 and one of 312.8 at index 37,890, and measures a plain serial loop as the reference. It then creates a ParallelMinSearch and a ParallelMaxSearch using the superclass type SensorSearchEngine, each with 4 worker threads, and starts them on two coordinator threads named MinSearch-Coordinator and MaxSearch-Coordinator so that both reductions run at the same time, joins both coordinators, and prints the global results and timings. After compiling with javac Q5/*.java and running java

Q5.SensorGridMonitorApp, the terminal showed that MinWorker-1 (segment [0..12500)) found -12.500 and MaxWorker-4 (segment [37500..50000)) found 312.800, exactly the segments that contain the planted indexes 12,345 and 37,890, the global minimum and peak were -12.500 and 312.800, and the line Matches serial reference: true confirmed that the multithreaded partitioning and reduction produce the same answer as the serial scan.

Question 6: Inventory Catalog Concurrency Benchmark

Source Code: InventoryCatalogManager.java

```
package Q6;

import java.util.Collections;
import java.util.HashMap;
import java.util.Map;
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.CountDownLatch;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
import java.util.concurrent.ThreadLocalRandom;
import java.util.function.Supplier;


public class InventoryCatalogManager
{

    private static final int THREADS = 10;
    private static final int TOTAL_OPERATIONS = 50_000;
    private static final int OPS_PER_THREAD = TOTAL_OPERATIONS / THREADS;
    private static final int PRODUCT_COUNT = 1_000;
    private static final int INITIAL_STOCK = 10;
    private static final int UPDATE_PERCENT = 30;   // 70% lookups, 30% stock updates
    private static final int WARMUP_ROUNDS = 3;
    private static final int MEASURED_ROUNDS = 5;

    private final String[] productIds = new String[PRODUCT_COUNT];

    public InventoryCatalogManager()
    {
        for (int i = 0; i < PRODUCT_COUNT; i++)
        {
            productIds[i] = "PROD-" + i;
        }
    }


    public static class BenchmarkResult
    {
        final String name;
        final double millis;
        final double opsPerSecond;

        BenchmarkResult(String name, double millis, double opsPerSecond)
        {
            this.name = name;
            this.millis = millis;
            this.opsPerSecond = opsPerSecond;
        }
    }
```

```
public BenchmarkResult benchmarkSynchronizedMap()
{
    return runBenchmark("Collections.synchronizedMap(HashMap)",
            () -> Collections.synchronizedMap(new HashMap<String, Integer>()));
}


public BenchmarkResult benchmarkConcurrentMap()
{
    return runBenchmark("ConcurrentHashMap",
            () -> new ConcurrentHashMap<String, Integer>());
}

private BenchmarkResult runBenchmark(String name, Supplier<Map<String, Integer>> mapFactory)
{
    for (int i = 0; i < WARMUP_ROUNDS; i++)
    {
        runOnce(mapFactory.get());
    }

    long totalNanos = 0;
    System.out.println("\n--- " + name + " ---");
    for (int r = 1; r <= MEASURED_ROUNDS; r++)
    {
        long nanos = runOnce(mapFactory.get());
        totalNanos += nanos;
        System.out.printf("  Round %d: %.3f ms%n", r, nanos / 1e6);
    }

    double avgNanos = (double) totalNanos / MEASURED_ROUNDS;
    double millis = avgNanos / 1e6;
    double opsPerSecond = TOTAL_OPERATIONS / (avgNanos / 1e9);

    System.out.printf("  Average execution time : %.3f ms%n", millis);
    System.out.printf("  Throughput             : %,.0f operations/second%n", opsPerSecond);
    return new BenchmarkResult(name, millis, opsPerSecond);
}


private long runOnce(Map<String, Integer> catalog)
{
    for (String id : productIds)
    {
        catalog.put(id, INITIAL_STOCK);
    }

    ExecutorService pool = Executors.newFixedThreadPool(THREADS);
    CountDownLatch ready = new CountDownLatch(THREADS);
    CountDownLatch startGate = new CountDownLatch(1);
    CountDownLatch done = new CountDownLatch(THREADS);
    long[] sold = new long[THREADS];
    long[] sinks = new long[THREADS];

    for (int t = 0; t < THREADS; t++)
    {
        final int id = t;
```

```
        pool.submit(() -> {
            ThreadLocalRandom rnd = ThreadLocalRandom.current();
            long[] successes = {0};
            long sink = 0;
            ready.countDown();
            try
            {

                startGate.await();       // all workers start together
            }
            catch (InterruptedException e)
            {
                Thread.currentThread().interrupt();
                done.countDown();
                return;
            }

            for (int i = 0; i < OPS_PER_THREAD; i++)
            {
                String product = productIds[rnd.nextInt(PRODUCT_COUNT)];
                if (rnd.nextInt(100) < UPDATE_PERCENT)
                {

                    catalog.compute(product, (k, stock) -> {
                        if (stock != null && stock > 0) {
                            successes[0]++;
                            return stock - 1;
                        }
                        return stock;
                    });
                }

                else
                {
                    Integer stock = catalog.get(product);   // lookup
                    if (stock != null)
                    {
                        sink += stock;
                    }
                }
            }
            sold[id] = successes[0];
            sinks[id] = sink;
            done.countDown();
        });
    }

    long elapsed;
    try
    {
        ready.await();
        long t0 = System.nanoTime();
        startGate.countDown();
        done.await();
        elapsed = System.nanoTime() - t0;
```

```
        }
        catch (InterruptedException e)
        {
            Thread.currentThread().interrupt();
            pool.shutdownNow();
            return 0;
        }
        pool.shutdown();

        //  check: stock removed from the catalog must equal successful updates
        long totalSold = 0;
        for (long s : sold)
        {
            totalSold += s;
        }
        long remaining = 0;
        for (Integer stock : catalog.values())
        {
            remaining += stock;
        }
        long expectedRemaining = (long) PRODUCT_COUNT * INITIAL_STOCK - totalSold;
        if (remaining != expectedRemaining)
        {
            throw new IllegalStateException("Inventory corrupted: expected "
                    + expectedRemaining + " but found " + remaining);
        }
        return elapsed;
    }

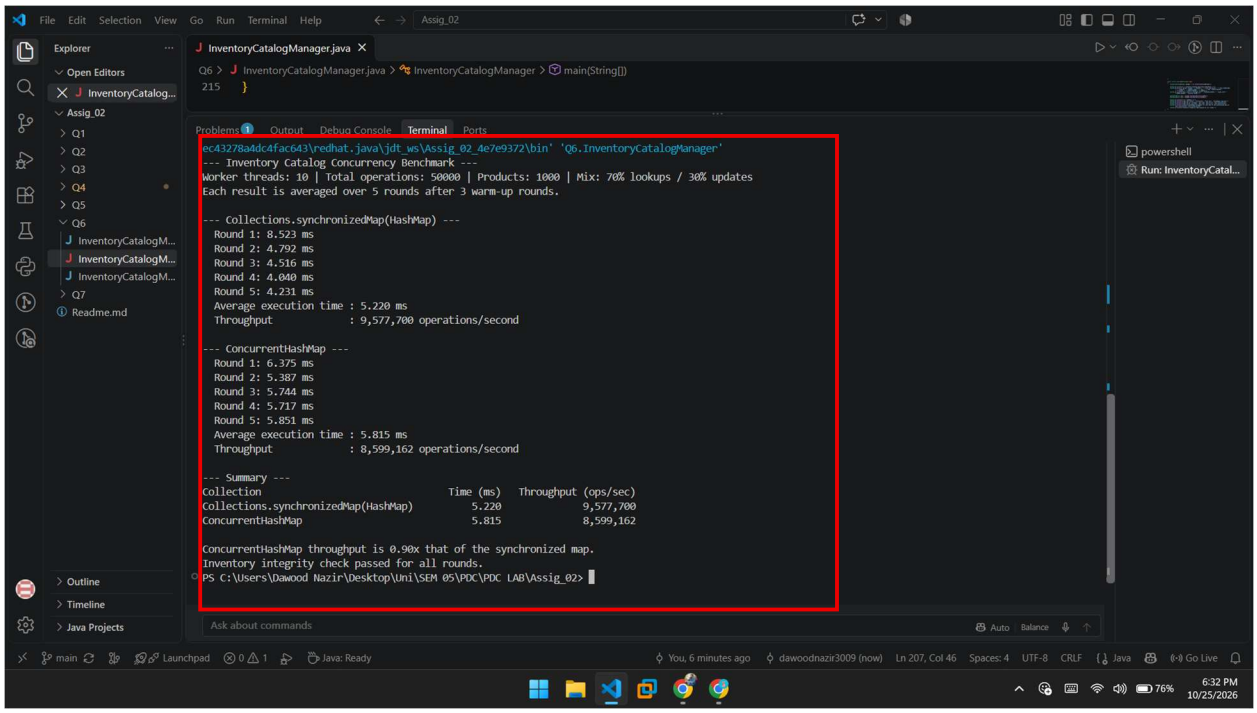
    public static void main(String[] args)
    {
        InventoryCatalogManager manager = new InventoryCatalogManager();

        System.out.println("--- Inventory Catalog Concurrency Benchmark ---");
        System.out.println("Worker threads: " + THREADS + " | Total operations: " + TOTAL_OPERATIONS
                + " | Products: " + PRODUCT_COUNT + " | Mix: " + (100 - UPDATE_PERCENT)
                + "% lookups / " + UPDATE_PERCENT + "% updates");
        System.out.println("Each result is averaged over " + MEASURED_ROUNDS + " rounds after "
                + WARMUP_ROUNDS + " warm-up rounds.");

        BenchmarkResult sync = manager.benchmarkSynchronizedMap();
        BenchmarkResult conc = manager.benchmarkConcurrentMap();

        System.out.println("\n--- Summary ---");
        System.out.printf("%-38s %12s %22s%n", "Collection", "Time (ms)", "Throughput (ops/sec)");
        System.out.printf("%-38s %12.3f %,22.0f%n", sync.name, sync.millis, sync.opsPerSecond);
        System.out.printf("%-38s %12.3f %,22.0f%n", conc.name, conc.millis, conc.opsPerSecond);
        System.out.printf("%nConcurrentHashMap throughput is %.2fx that of the synchronized map.%n",
                conc.opsPerSecond / sync.opsPerSecond);
        System.out.println("Inventory integrity check passed for all rounds.");
    }
}
```

Execution Screenshot



Approach and Explanation

I solved this task with one class, InventoryCatalogManager.java inside package Q6, that benchmarks two thread-safe map implementations under the same workload. The class defines constants for 10 worker threads, 50,000 total operations (5,000 per thread), 1,000 products, an initial stock of 10 per product, a 30% update and 70% lookup mix, 3 warm-up rounds and 5 measured rounds; its constructor fills a String[] with the product IDs PROD-0 to PROD-999, and a static nested class BenchmarkResult stores the name, the average time in milliseconds and the throughput in operations per second. The two public methods benchmarkSynchronizedMap() and benchmarkConcurrentMap() both call the same runBenchmark(name, Supplier<Map<String, Integer>>) method, passing a factory that returns Collections.synchronizedMap(new HashMap<>()) or new ConcurrentHashMap<>(), so the only difference between the two runs is the collection. Inside runOnce() the map is filled with the initial stock, an ExecutorService from Executors.newFixedThreadPool(10) starts the workers, and three CountDownLatch objects (ready, startGate and done) make all workers begin at the same moment so that System.nanoTime() measures only the concurrent work. Each worker uses ThreadLocalRandom to pick a random product and, when nextInt(100) is below 30, performs a stock update with catalog.compute(), which atomically decrements the stock only if it is greater than zero and counts the successful sale, while the remaining operations perform a catalog.get() lookup. After every round the program verifies integrity by checking that the remaining stock plus the total sold equals 1,000 x 10 and throws an IllegalStateException otherwise. The synchronized map guards every call with a single lock, so the threads queue behind each other, whereas ConcurrentHashMap uses finer-grained

locking and non-blocking reads so that threads working on different buckets can proceed in parallel. After compiling with javac Q6/*.java and running java Q6.InventoryCatalogManager, the terminal listed the five round times of each map, their average execution time and throughput in a summary table, the throughput ratio of ConcurrentHashMap against the synchronized map, and the message Inventory integrity check passed for all rounds, which verified that no stock update was lost under either collection.

Question 7: Cache Locality Benchmark (Row-Major vs Column-Major)

Source Code: CacheLocalityBenchmark.java

```
package Q7;

public class CacheLocalityBenchmark
{
   private static final int N = 2000;
   private static final int ITERATIONS = 10;
   private static final int WARMUP = 2;

   private static final double[][] matrix = new double[N][N]; //2-D array


   static double traverseRowMajor()
   {

      double sum = 0.0;
      for ( int i = 0; i < N; i++)
      {
         for ( int j = 0; j < N; j++)
         {
            sum += matrix [i][j];

         }
      }
      return sum;
   }


   static double traverseColumnMajor()
   {
      double sum = 0.0;
      for ( int j = 0; j < N; j++)
      {
         for ( int i = 0; i < N; i++)
         {
            sum += matrix [i][j];

         }
      }
      return sum;
   }

   public static void main(String[] args)
   {
      for ( int i = 0; i < N; i++)
      {
         for ( int j = 0; j < N; j++)
         {
             matrix [i][j] = (i + j) % 10;

         }
      }
```

```
System.out.println("--- Cache Locality Benchmark --- ");
System.out.print("Matrix: " + N + " x " + N + "doubles (");
System.out.println((N * N * 8L / (1024*1024)) + " MB) | Iterations: " + ITERATIONS);

double warmsink = 0;
for (int w = 0; w < WARMUP; w++)
{

   warmsink += traverseRowMajor();
   warmsink += traverseColumnMajor();
}

double[] rowms = new double[ITERATIONS];
double[] colms = new double[ITERATIONS];
double RowSum = 0, ColSum = 0;

for (int it = 0; it < ITERATIONS; it++)
{
   long t0 = System.nanoTime();
   RowSum = traverseRowMajor();
rowms[it] = (System.nanoTime() - t0) / 1e6;

   t0 = System.nanoTime();
   ColSum = traverseColumnMajor();
   colms[it] = (System.nanoTime() - t0) / 1e6;
}

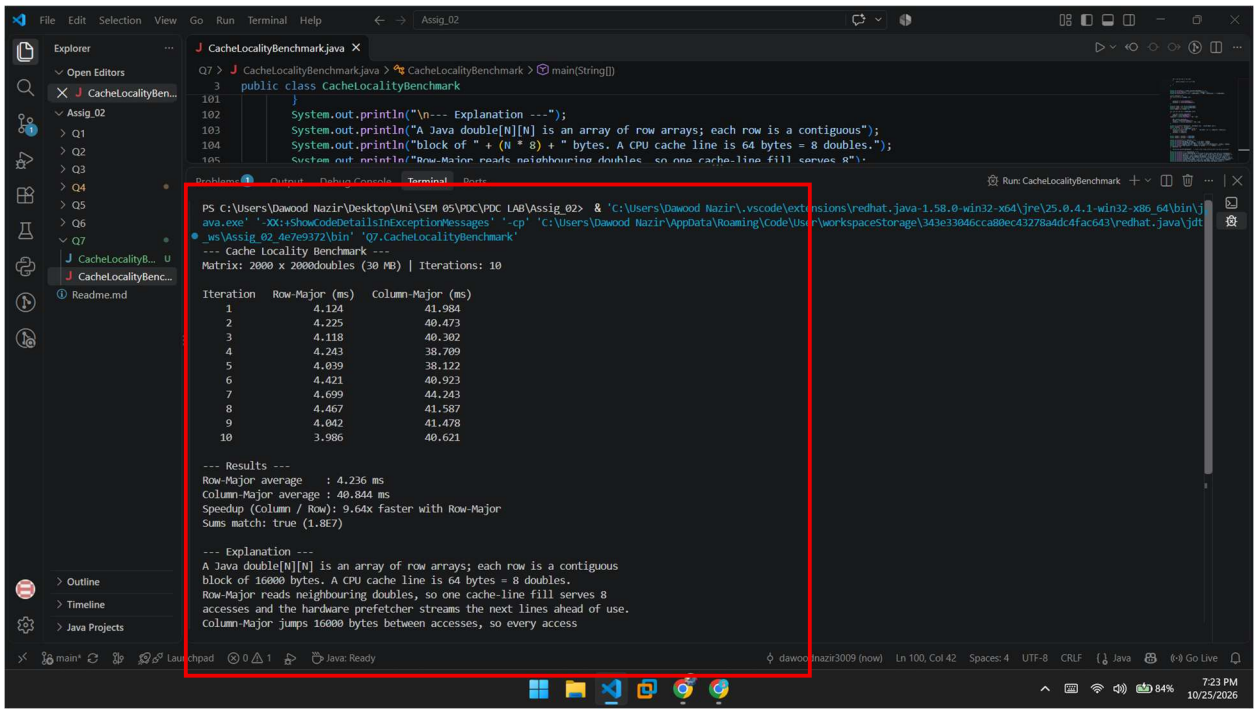
System.out.println("\nIteration   Row-Major (ms)   Column-Major (ms)");
double rowTotal = 0, colTotal = 0;
for (int it = 0; it < ITERATIONS; it++) {
   System.out.printf("%5d      %12.3f     %14.3f%n", it + 1, rowms[it], colms[it]);
   rowTotal += rowms[it];
   colTotal += colms[it];
}

double rowAvg = rowTotal / ITERATIONS;
double colAvg = colTotal / ITERATIONS;

System.out.println("\n--- Results ---");
System.out.printf("Row-Major average    : %.3f ms%n", rowAvg);
System.out.printf("Column-Major average : %.3f ms%n", colAvg);
System.out.printf("Speedup (Column / Row): %.2fx faster with Row-Major%n", colAvg / rowAvg);
System.out.println("Sums match: " + (RowSum == ColSum) + " (" + RowSum + ")");
if (warmsink < 0)
{
   System.out.println(warmsink);   // never true; keeps warm-up work from being discarded
}
System.out.println("\n--- Explanation ---");
System.out.println("A Java double[N][N] is an array of row arrays; each row is a contiguous");
System.out.println("block of " + (N * 8) + " bytes. A CPU cache line is 64 bytes = 8 doubles.");
System.out.println("Row-Major reads neighbouring doubles, so one cache-line fill serves 8");
System.out.println("accesses and the hardware prefetcher streams the next lines ahead of use.");
System.out.println("Column-Major jumps " + (N * 8) + " bytes between accesses, so every access");
```

```
        System.out.println("touches a different cache line (and page), using 1 of 8 doubles loaded");
        System.out.println("and causing many more cache and TLB misses and memory stalls.");
    }
}
```

Execution Screenshot



Approach and Explanation

I solved this task with a single class, CacheLocalityBenchmark.java inside package Q7, that profiles how memory access order affects speed. The class declares the constants N = 2000, ITERATIONS = 10 and WARMUP = 2 and a static double[N][N] matrix, which is filled inside main() with two nested loops using the value (i + j) % 10 so that the data is small and the sums are exact. The static method traverseRowMajor() uses i as the outer loop and j as the inner loop to add matrix[i][j], so it reads neighbouring elements of the same row array, while traverseColumnMajor() swaps the loops (j outer, i inner) and therefore jumps from one row array to the next on every access. Both methods return the same sum. Before measuring, main() runs the two traversals twice as a warm-up and keeps the result in a warmsink variable that is checked later so that the JIT compiler cannot remove the work; it then runs 10 iterations, measuring each traversal with System.nanoTime(), converting the difference to milliseconds and storing it in the rowms and colms arrays, and finally prints a per-iteration table, the two averages and the ratio colAvg / rowAvg as the speedup of row-major traversal. A Java double[N][N] is an array of row arrays in which each row is a contiguous block of 16,000 bytes, and a 64-byte cache line holds 8 doubles, so row-major traversal uses all 8 doubles of every cache line that is loaded (spatial locality) and lets the hardware prefetcher stream the next lines, whereas column-major traversal skips 16,000 bytes between accesses, touches a new cache line and memory page almost every

time, and causes many more cache and TLB misses. After compiling with javac Q7/*.java and running java Q7.CacheLocalityBenchmark on the 2000 x 2000 matrix (30 MB), the terminal showed that the row-major average time was several times smaller than the column-major average time, and the line Sums match: true (1.8E7) confirmed that both traversals computed the same result, so the difference in time came only from cache locality and not from different work.

GitHub Repository and Submission Details

Repository Link

GitHub Repository: https://github.com/dawoodnazir3009/PDC-Lab-Assignment-2-Fall-2026

The complete source code of Questions 1 to 7 (folders Q1 to Q7), the README documentation and the full commit history (branch main, 58 commits with readable messages) are available in the repository above. The same Java source files are also attached with this report in the submitted zip file.

GitHub Repository Screenshot

